

AI-POWERED RESUME SCREENER

PROJECT REPORT

Student Name: Appikarla Indu

Roll Number: 24NP1A0502

Department: Computer Science and Engineering

College/Institution: Vijaya Institute of Technology for Womens

1. ABSTRACT

The AI-Powered Resume Screener is a web-based application that automates resume screening and candidate ranking. It accepts resumes in PDF, DOCX, and TXT formats, extracts important candidate information, compares resumes with a given job description using TF-IDF and cosine similarity, detects relevant skills, and produces a ranked list of candidates.

2. INTRODUCTION

Recruiters often receive a large number of resumes for a single job opening. Manual screening can be time-consuming and may lead to inconsistent evaluation. This project applies Artificial Intelligence and Natural Language Processing techniques to automate the initial screening process.

3. PROBLEM STATEMENT

Traditional resume screening requires recruiters to read and compare many documents manually. This increases time and effort and makes it difficult to consistently compare candidates.

4. OBJECTIVES

- Extract candidate details automatically.
- Support PDF, DOCX, and TXT files.
- Compare resumes with job descriptions using TF-IDF.
- Calculate match scores using cosine similarity.
- Detect matched and missing skills.
- Rank candidates.
- Provide a simple web interface.

5. EXISTING SYSTEM

Recruiters manually open each resume, identify skills and experience, compare them with job requirements, and prepare a shortlist. This process is slow, repetitive, and difficult to scale.

6. PROPOSED SYSTEM

Resumes are uploaded through a Flask web application. Text is extracted, candidate information and skills are detected, resume content is compared with the job description, and candidates are ranked by match score.

7. SYSTEM ARCHITECTURE

Resume Upload → Document Text Extraction → Candidate Information Extraction → Skill Detection → TF-IDF Vectorization → Cosine Similarity → Match Score → Candidate Ranking → Screening Result.

8. TECHNOLOGIES USED

Python, Flask, spaCy, Scikit-learn, pypdf, python-docx, HTML/CSS, and TF-IDF based Natural Language Processing.

9. FEATURES & FUNCTIONAL MODULES

- Multiple resume upload.

- PDF, DOCX, and TXT parsing.
- Candidate information extraction.
- Job description and required-skills input.
- TF-IDF resume matching.
- Match score calculation.
- Matched and missing skill identification.
- Candidate ranking.
- Selected/Not Selected result.

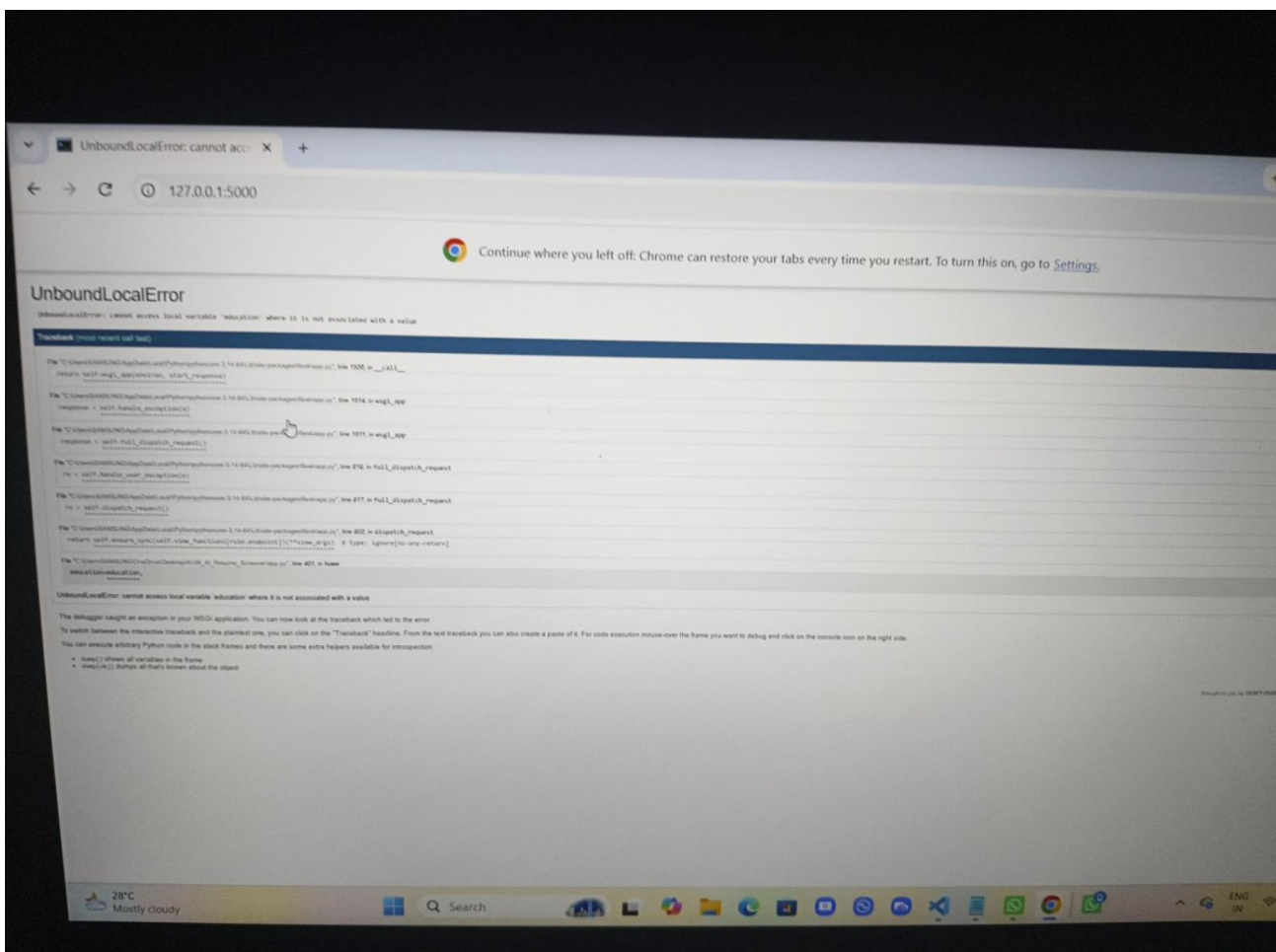
10. IMPLEMENTATION

The application is implemented using Python and Flask. Document parsers read uploaded resume files. Scikit-learn converts resume and job-description text into TF-IDF vectors and cosine similarity calculates the matching score. The Flask interface displays screening results and ranking.

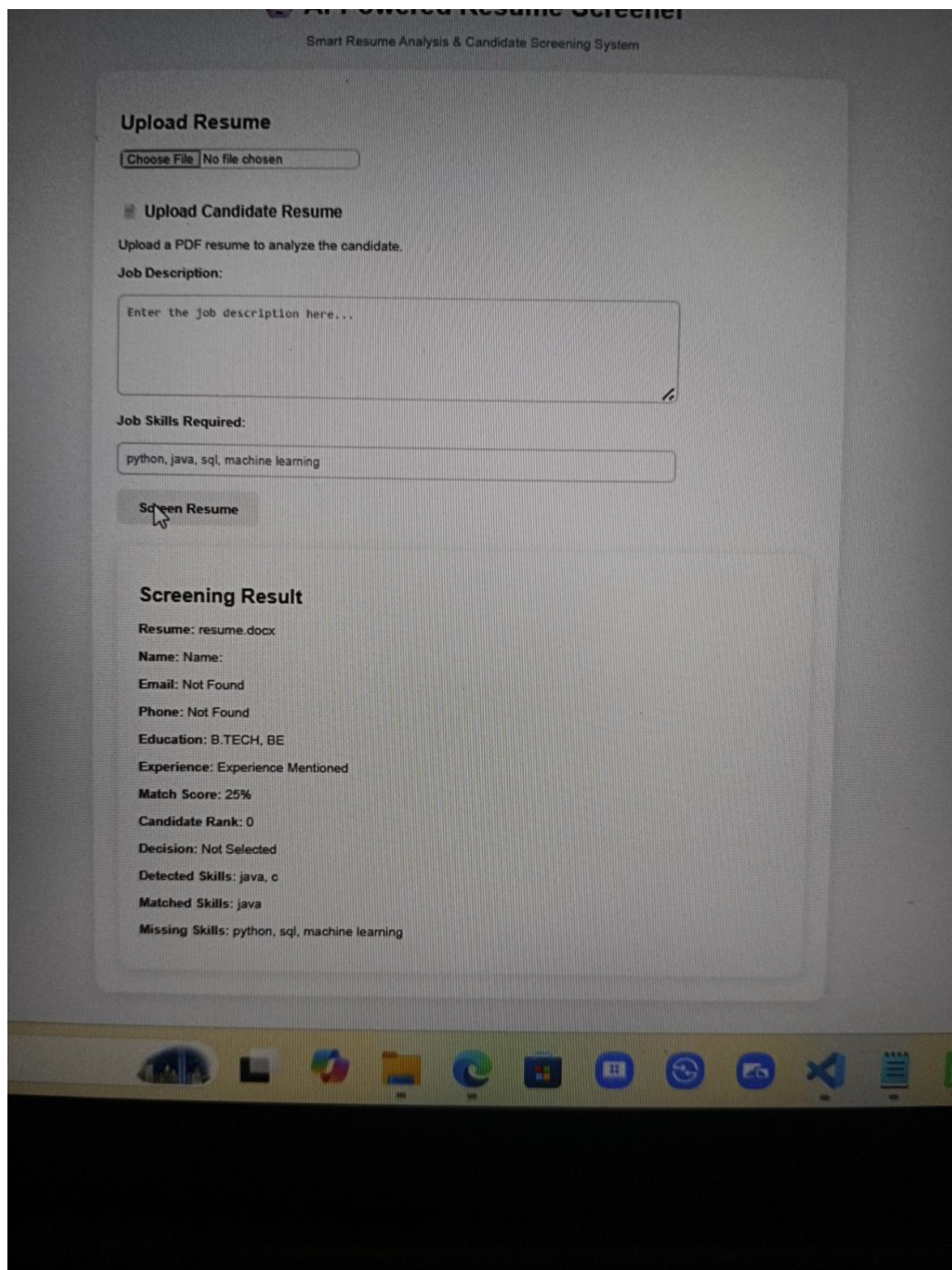
11. SYSTEM OUTPUT

The application displays candidate details, match percentage, candidate rank, screening decision, detected skills, matched skills, and missing skills. Multiple resumes are ranked from the highest match score to the lowest.

12. PROJECT SCREENSHOTS



Home Page



Resume Upload + Job Description

```
130     file_path = os.path.join(UPLOAD_FOLDER, safe_name)
131
132     file.save(file_path)
133
134     resume_text = extract_text(file_path)
135
136
137     required_skills = [
138         skill.strip().lower()
139         for skill in job_skills.split(",")
140         if skill.strip()
141     ]
142
143     for skill in required_skills:
```

Problems Output Debug Console Terminal Ports

IndentationError: unindent does not match any outer indentation level
PS C:\Users\SAMSUNG\OneDrive\Desktop\AI-09_AI_Resume_Screener> python app.py
File "C:\Users\SAMSUNG\OneDrive\Desktop\AI-09_AI_Resume_Screener\app.py", line 1
if score >= 70:
IndentationError: unexpected indent
PS C:\Users\SAMSUNG\OneDrive\Desktop\AI-09_AI_Resume_Screener> python app.py
File "C:\Users\SAMSUNG\OneDrive\Desktop\AI-09_AI_Resume_Screener\app.py", line 1
if score >= 70:
IndentationError: unexpected indent
PS C:\Users\SAMSUNG\OneDrive\Desktop\AI-09_AI_Resume_Screener> █

Screening Result + Ranked Candidates

13. TEST CASES & RESULTS

PDF resume upload — Passed.

DOCX resume upload — Passed.

TXT resume upload — Passed.

Job description and skills input — Passed.

Match score calculation — Passed.

Multiple candidate ranking — Passed.

Screening decision display — Passed.

14. ADVANTAGES

- Reduces manual screening time.
- Provides consistent first-level comparison.
- Supports multiple document formats.
- Automatically ranks candidates.
- Easy to use through a web interface.

15. FUTURE ENHANCEMENTS

- Use transformer-based language models for deeper semantic matching.
- Add a recruiter dashboard and database.
- Improve entity extraction.
- Add configurable scoring rules.
- Provide analytics and downloadable reports.

16. LEARNING OUTCOMES

- Information Extraction – extracting useful candidate details from documents.
- TF-IDF Matching – measuring similarity between resumes and job descriptions.
- Document Parsing – processing PDF, DOCX, and TXT files.
- AI-Based Resume Screening – applying automated analysis and candidate ranking.

17. DELIVERABLES

Web application, Python source code, sample resumes, project report, presentation, and demonstration of the working screening workflow.

18. CONCLUSION

The AI-Powered Resume Screener demonstrates how Artificial Intelligence, Natural Language Processing, and machine-learning based text similarity can automate the initial resume screening process. The system extracts information, compares resumes with job requirements, calculates match scores, and ranks candidates.

19. REFERENCES

Python documentation; Flask documentation; Scikit-learn documentation; spaCy documentation; pypdf documentation; python-docx documentation.